

Chapter 22

22.1 Recovery Concepts

When a database system fails (crash, power loss, error), the DBMS must restore the database to a correct and consistent state. To achieve this, the DBMS maintains a special file called the **log**. The log records all actions of transactions (before and after values), so the system can redo or undo actions during recovery.

22.1.1 Recovery Outline

There are two primary types of failures:

1. Catastrophic Failure

- Examples: disk crash, fire, complete system corruption.
- The database on disk is destroyed.

Recovery actions:

1. Restore the database from a backup copy (e.g., tape or offline storage).
2. Reapply all committed transactions using log records (REDO).

Note: UNDO is not required because the restored database is an old consistent copy with no partial transactions.

2. Non-Catastrophic Failure

- Examples: system crash, power outage, OS crash, deadlock rollback, transaction error.
- Database on disk is still intact, but some transactions remain incomplete.

Recovery actions:

- If a committed transaction's updates are not yet on disk → perform **REDO**.
- If an uncommitted transaction wrote some values to disk → perform **UNDO**.

Thus, the log is used to determine:

Which transactions should be undone? Which should be redone?

Types of Recovery Policies

1. Deferred Update (NO-UNDO / REDO)

Idea: Do not write updates to the database until the transaction commits.

- All updates remain in memory (buffer).
- Only log records are written to disk before commit.

If a transaction fails before commit:

No UNDO needed (nothing was written to disk).

If a committed transaction's updates were not flushed to disk:

REDO is required.

2. Immediate Update (UNDO / REDO)

Idea: Updates may be written to disk before commit, but the log must contain:

BFIM (Before Image), AFIM (After Image)

If a transaction fails before commit:

UNDO using BFIM.

If it committed but some writes are missing:

REDO using AFIM.

This policy is used in most modern DBMS.

Idempotent UNDO and REDO

An operation is **idempotent** if performing it multiple times has the same effect as performing it once.

Why needed? If the system crashes during recovery, recovery must be safe to execute repeatedly.

22.1.2 DBMS Buffer Cache

To speed up processing, DBMS stores part of the database in buffer pages in RAM.

Each buffer page contains:

- **Dirty bit:** set to 1 if the page was modified.
- **Read bit/Pin bit:** set to 1 if the page should not be written to disk (e.g., not committed yet).

If the buffer is full and a new page is needed, the DBMS may write a dirty page to disk.

In-Place Updating vs Shadowing

In-Place Updating:

- Write directly to the same disk location.
- Requires BFIM and AFIM for UNDO/REDO.

Shadowing:

- Write updated pages to new locations on disk.
- Maintains old and new versions.
- Rarely used in practice.

22.1.3 Write-Ahead Logging (WAL)

Rule:

The log must be written to disk *before* the actual database page is written.

Purpose: Ensures that crash recovery has all necessary information for UNDO or REDO.

STEAL / NO-STEAL and FORCE / NO-FORCE Policies

STEAL

A dirty page may be written to disk before its transaction commits.

⇒ UNDO required

NO-STEAL

Pages modified by uncommitted transactions cannot be written to disk.

⇒ No UNDO needed

FORCE

All pages of a committing transaction are written to disk at commit time.

⇒ No REDO needed

NO-FORCE

Pages of committed transactions may still be in memory.

⇒ REDO required

Most DBMS use:

STEAL + NO-FORCE

This requires both UNDO and REDO but offers the best performance.

22.1.4 Checkpoints

Problem: The log may grow very large. Reading the entire log during recovery is time-consuming.

Solution: At intervals, the DBMS writes:

[CHECKPOINT, active transactions]

and flushes dirty pages to disk.

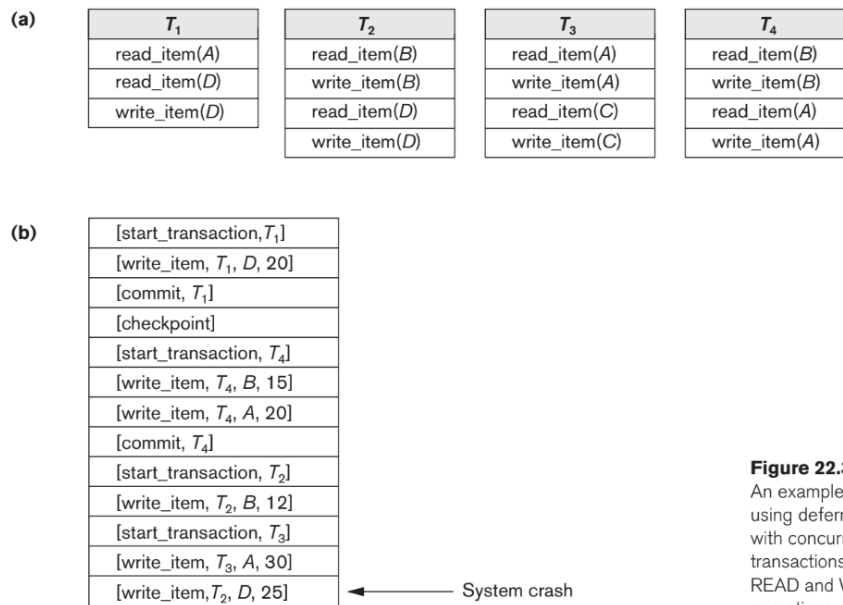
During recovery:

- Transactions committed before the checkpoint → safe, no REDO.
- Only transactions active after the checkpoint need UNDO/REDO.

Fuzzy Checkpointing

To avoid stopping the system:

1. Write [begin_checkpoint] to the log.
2. Continue normal processing.
3. Flush dirty pages in the background.
4. Write [end_checkpoint] when done.



T_2 and T_3 are ignored because they did not reach their commit points.
 T_4 is redone because its commit point is after the last system checkpoint.

Figure 22.3
 An example of recovery using deferred update with concurrent transactions. (a) The READ and WRITE operations of four transactions. (b) System log at the point of crash.

Explanation of Recovery using Deferred Update

Based on Figure 22.3, the system is utilizing the **Deferred Update** (NO-UNDO/REDO) protocol. In this protocol, updates are recorded in the log but are not applied to the physical database on disk until the transaction reaches its commit point.

System Status at Crash

The system log shows the sequence of operations up to the moment of the crash. We analyze the timeline relative to the [checkpoint] entry.

- **Completed Transactions:** Transaction T_1 and T_4 have written a [commit, ...] entry to the log.
- **Active Transactions:** Transactions T_2 and T_3 have started and written log entries but have *not* committed before the system crash occurred.
- **The Checkpoint:** A [checkpoint] operation was performed after T_1 committed but before T_4 committed.

Recovery Logic

When the system restarts, the recovery manager performs the following actions based on the Deferred Update rules:

A. Identify Transaction Lists

The system scans the log to populate two lists:

1. **Committed Transactions List:** T_4 (committed after the last checkpoint). Note: T_1 is also committed, but it finished *before* the checkpoint, so its data is already safely written to the disk.
2. **Active (Failed) Transactions List:** T_2 and T_3 .

Applying Recovery Operations

- **For T_2 and T_3 (IGNORE):** Since the Deferred Update protocol does not write to the database until commit, the changes made by T_2 and T_3 (e.g., T_2 writing $B = 12$, T_3 writing $A = 30$) currently exist only in the log or temporary buffers. They never touched the permanent database. Therefore, no **UNDO** is required. These transactions are simply ignored.
- **For T_1 (NO ACTION):** T_1 committed before the checkpoint. This guarantees its updates (writing $D = 20$) were flushed to the database disk during the checkpoint process. It does not need to be redone.
- **For T_4 (REDO):** T_4 committed, but it did so *after* the last checkpoint. There is no guarantee its changes reached the disk before the crash. Therefore, the system must **REDO** the operations of T_4 .

Final Result

The recovery process re-applies the write operations from T_4 :

Write(B) $\leftarrow$ 15

Write(A) $\leftarrow$ 20

The writes from T_2 and T_3 are discarded. The database is restored to a consistent state including T_1 and T_4 .

22.1.5 Cascading Rollback

If:

T_1 writes X and T_2 reads X and T_1 fails

then T_2 must also be rolled back.

To avoid this, DBMS use **strict schedules**:

A transaction cannot read values written by uncommitted transactions.

Thus:

Read operations need not be logged in modern systems.

22.1.6 Actions That Do Not Affect the Database

Examples:

- Printing a report
- Displaying results to the user
- Sending an email or notification

These should only occur **after commit**, otherwise the user might see output based on uncommitted work.

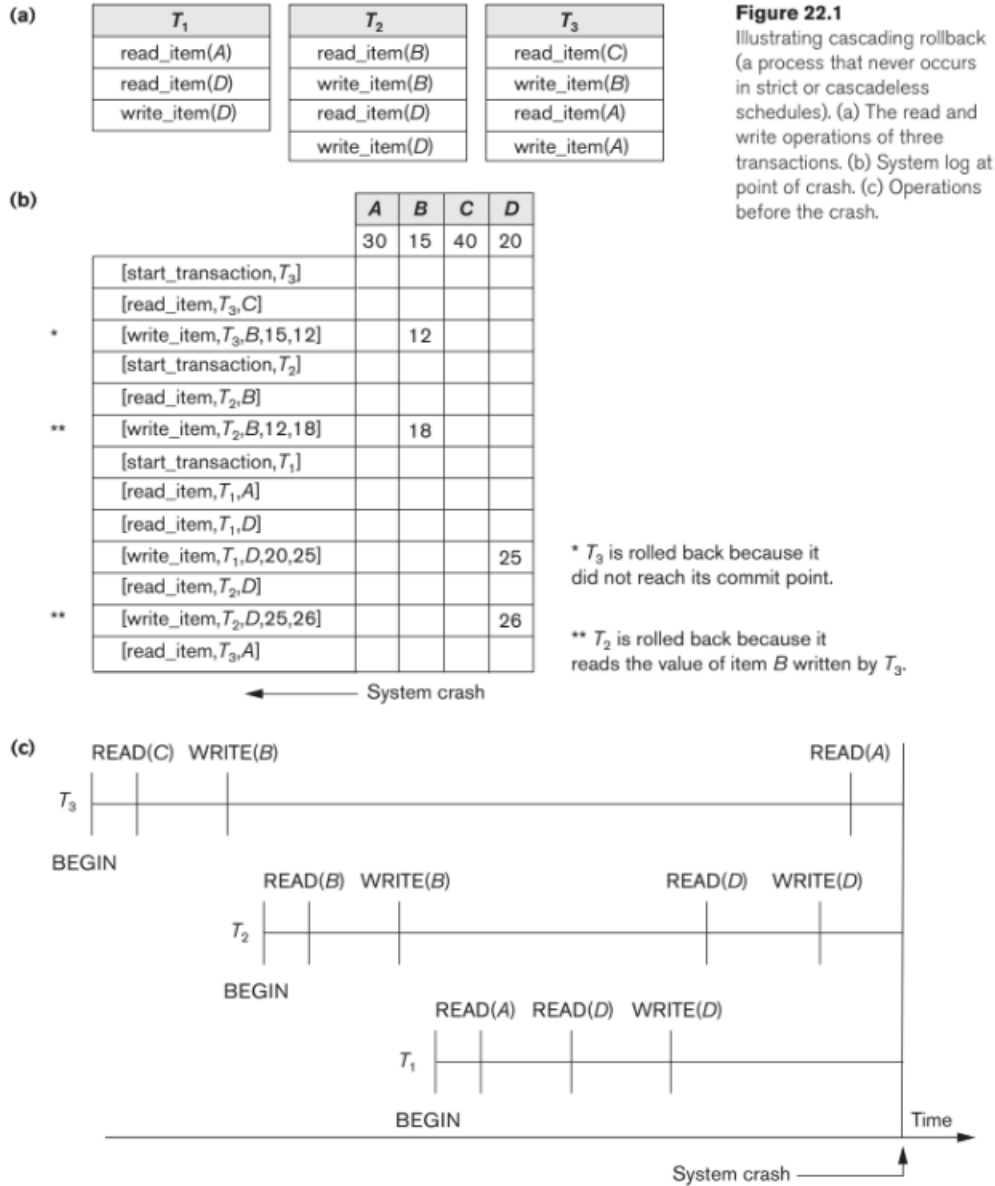
DBMS therefore queues such outputs until the transaction commits.

22.1.5 Transaction Rollback and Cascading Rollback

In database systems, when a transaction fails after updating the database but before committing, it must be rolled back to restore the database to a consistent state. Rollback ensures that changes made by a failed transaction do not persist in the database.

If another transaction reads data that was written by a transaction that is later rolled back, this second transaction must also be rolled back. This can lead to **cascading rollback**, where multiple dependent transactions are undone.

Example Scenario



Consider three transactions T_1 , T_2 , and T_3 operating on data items A , B , C , and D . The operations are shown in Table.

Cascading Rollback

- Transaction T_3 has not committed at the time of crash, so it must be rolled back. This restores the old value of B .
- Transaction T_2 read B written by T_3 , so T_2 must also be rolled back, restoring B and D to their previous values.
- Only **write** operations need to be undone; **read** operations are logged only to detect cascading rollback.

Conclusion

Cascading rollback occurs when one failed transaction forces other dependent transactions to be rolled back. Practical recovery protocols avoid cascading rollback by enforcing strict or cascadeless schedules. In such protocols, it is not necessary to log **read** operations.

22.2 NO-UNDO/REDO Recovery (Deferred Update)

Example: Bank Transactions

Transaction	Operation	Disk Status
T1	Withdraw \$100 from A	Buffer only, no disk change
T2	Deposit \$50 to B	Buffer only, no disk change
T1 commits	Write T1 changes to disk	Disk updated for T1
T2 commits	Write T2 changes to disk	Disk updated for T2

Recovery Timeline (Deferred Update)

Time	Transaction Actions	Disk Changes	Recovery Action
t1	T1 writes to buffer	No	-
t2	T2 writes to buffer	No	-
t3	T1 commits	Apply T1 to disk	REDO T1 if crash occurs
t4	Crash	-	REDO T1 only
t5	T2 commits	Apply T2 to disk	REDO T2 if needed

22.3 Immediate Update Recovery

Example: Bank Transactions

Transaction	Operation	Disk Status
T1	Withdraw \$100 from A	Disk updated immediately, UNDO log saved
T2	Deposit \$50 to B	Disk updated immediately, UNDO log saved
Crash before commit	-	Disk may have partial changes
T1 commits	-	REDO may reapply committed updates

Recovery Timeline (Immediate Update)

Time	Transaction Actions	Disk Changes	Recovery Action
t1	T1 writes to disk	Disk updated	UNDO T1 if crash before commit
t2	T2 writes to disk	Disk updated	UNDO T2 if crash before commit
t3	Crash occurs	Partial changes may exist	UNDO uncommitted transactions
t4	T1 commits	Changes are committed	REDO if needed

Comparison Table

Feature	Deferred Update	Immediate Update
Database updated	At commit	Immediately
Log type	REDO only	UNDO (and sometimes REDO)
Rollback needed	No	Yes, using UNDO logs
Buffer policy	No-steal	Can be steal / no-force
Best for	Short transactions	Transactions requiring quick updates